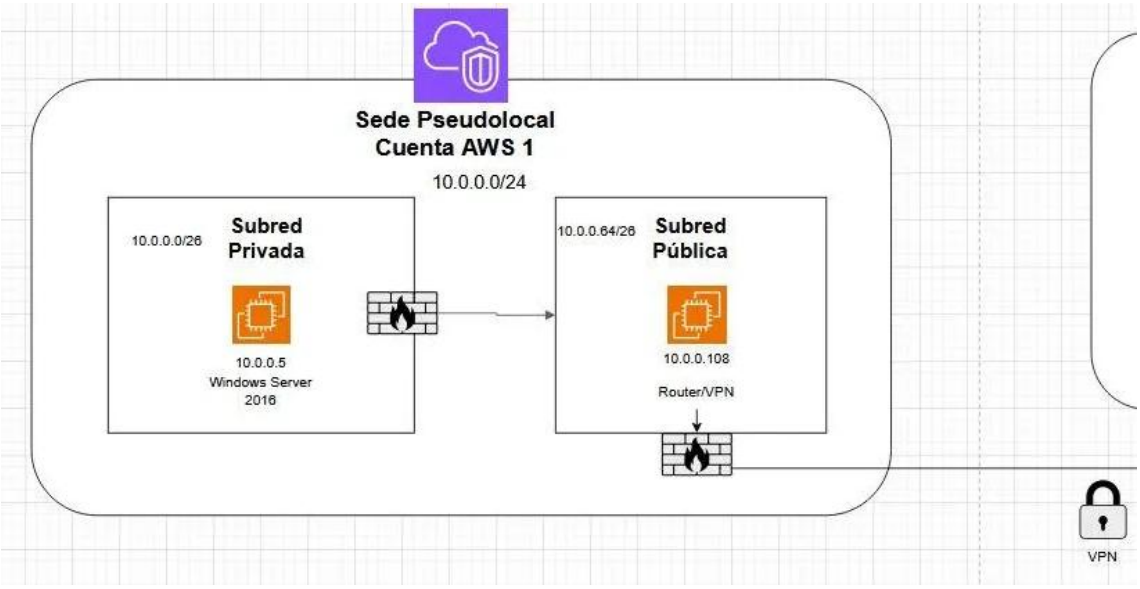
Elemento	Nombre en AWS	IP Privada	IP Pública	Función
VPC	VPC-Sede_Pseudolocal_Proyecto	10.0.0.0/24	-	Red General
Subred Pública	Red pública pseudolocal	10.0.0.64/26	-	Zona DMZ
Subred Privada	Red privada pseudolocal	10.0.0.0/26	-	Zona Segura
Router/VPN	VPN_NAT-router	**10.0.0.108**	100.53.224.6 (cambia)	Salto SSH / Gateway
Servidor AD	AD_Server16	10.0.0.50	No tiene	Controlador Dominio



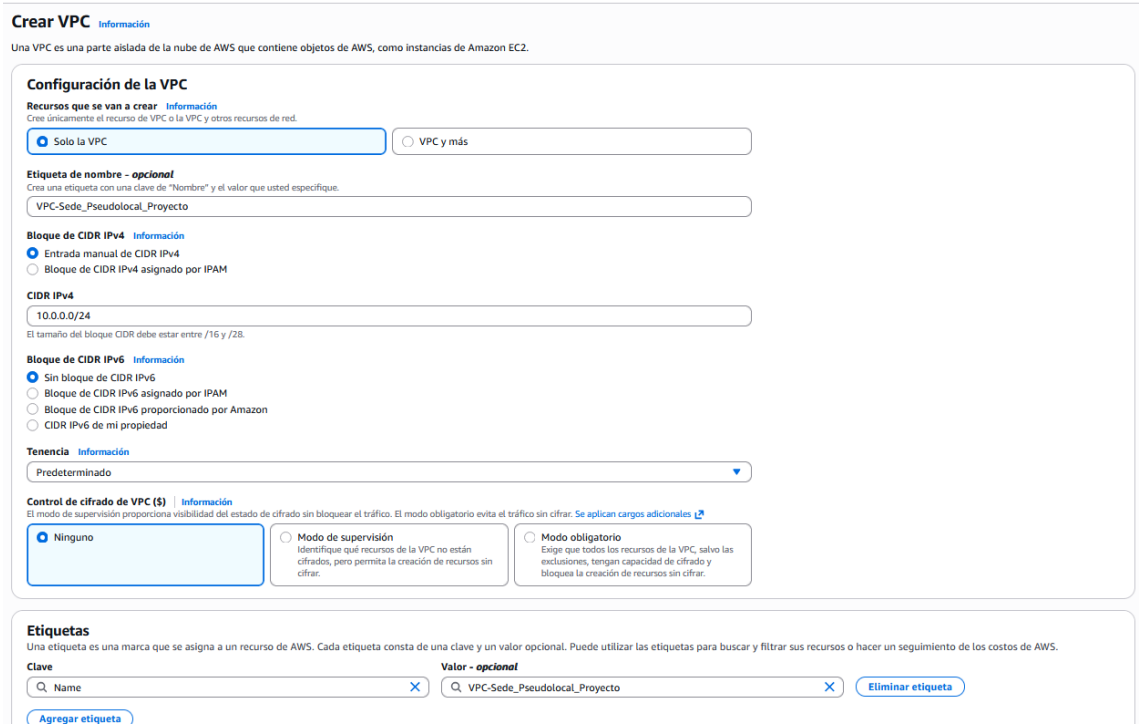
Segregación de red: **Hemos implementado** una división mediante subredes /26 para limitar el radio de un posible ataque, aislando el Directorio Activo de la subred con acceso directo a internet.

Bastion Host (Salto SSH): **Hemos decidido** no asignar IP pública al servidor Windows por seguridad. El acceso se realiza mediante un túnel SSH cifrado que mapea el puerto local 3390 al puerto RDP 3389 del servidor interno.

DNS de Bucle Local: **Hemos forzado** el uso de 127.0.0.1 en el Windows Server para asegurar que el rol de Active Directory gestione todas las peticiones de resolución del dominio proyectoasirG1.local.

Captura 1: Configuración de la VPC

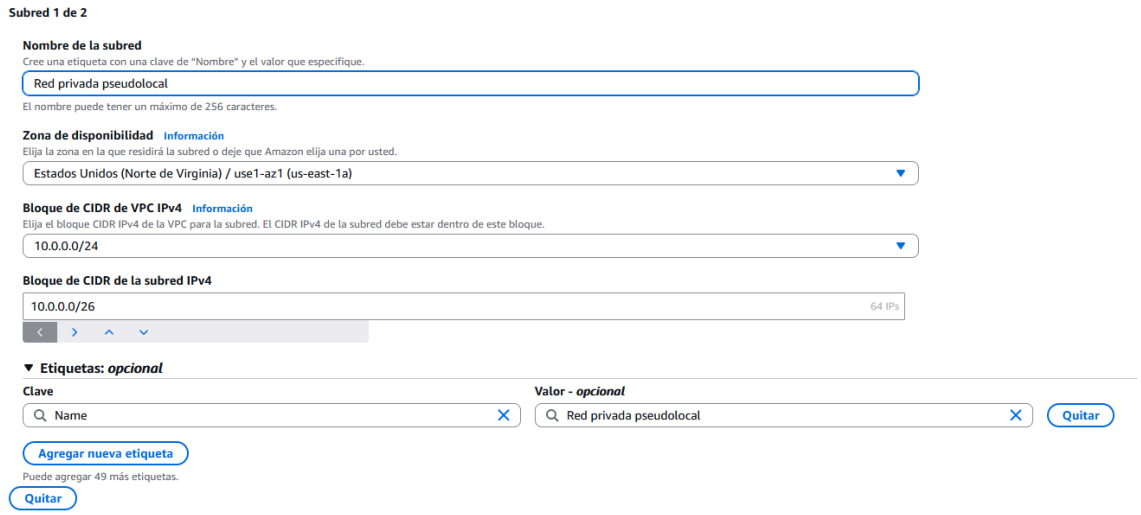
En este primer paso, hemos definido el entorno virtual donde vive todo el proyecto. Hemos creado una VPC denominada VPC-Sede_Pseudolocal_Proyecto con el rango 10.0.0.0/24, lo que nos da un direccionamiento de 256 IPs privadas. Hemos optado por la configuración manual para asegurar que cada componente se ajuste exactamente a nuestro diseño de red.

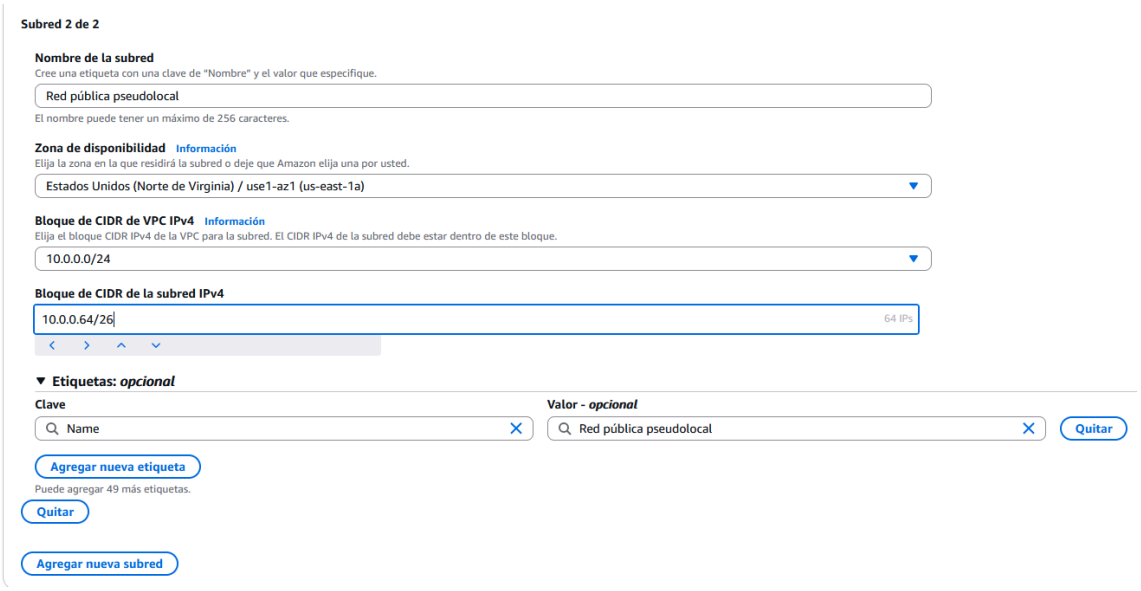


Capturas 2 y 3: Segmentación en Subredes

Hemos dividido la red en dos segmentos para separar los servicios según su nivel de exposición:

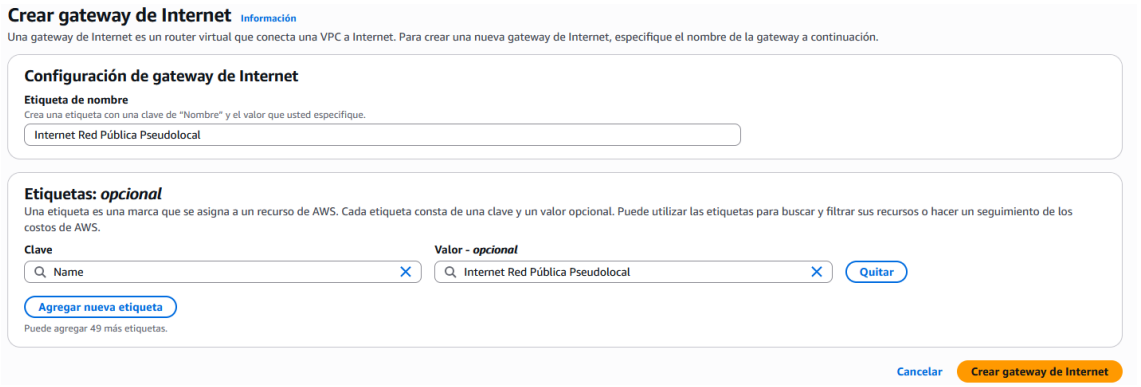
- **Red privada pseudolocal**: **Hemos asignado** el rango 10.0.0.0/26. Aquí es donde **hemos ubicado** el servidor de Directorio Activo para que no sea accesible directamente desde el exterior.
- **Red pública pseudolocal**: **Hemos usado** el rango 10.0.0.64/26. En esta subred **hemos instalado** el servidor Ubuntu que funcionará como pasarela a internet.

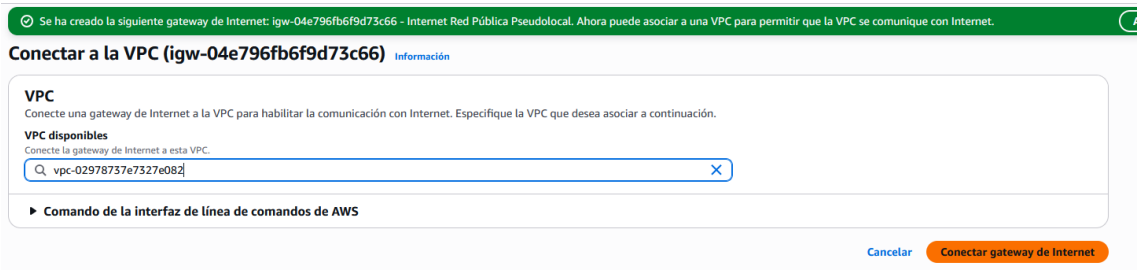




Capturas 4 y 5: Conexión con el exterior (Internet Gateway)

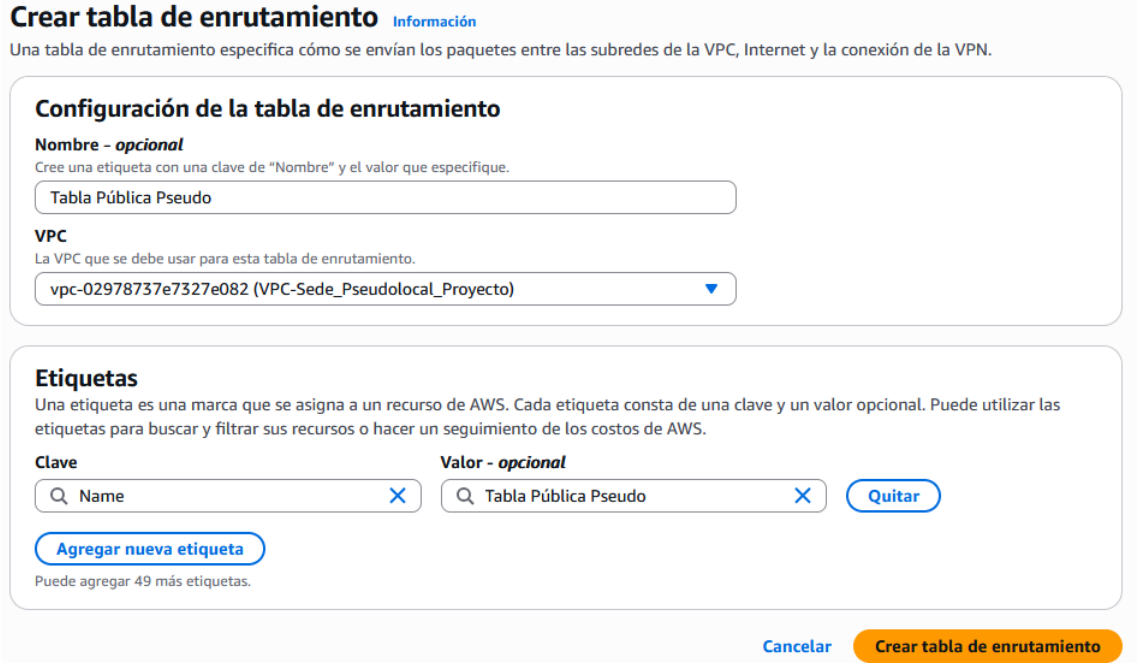
Para que nuestra red tenga salida a internet, **hemos creado** un **Internet Gateway** y lo **hemos vinculado** a nuestra VPC. Sin este paso, las máquinas estarían totalmente aisladas y no podríamos conectarnos remotamente ni descargar el software necesario.

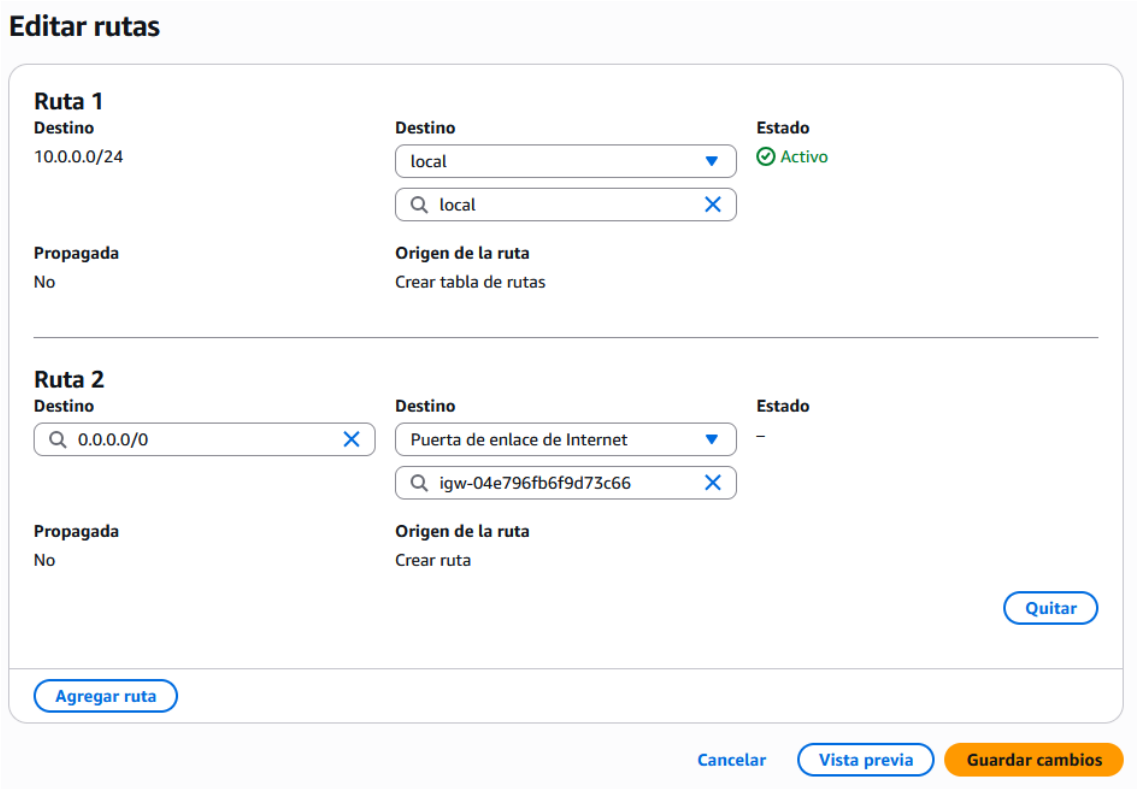




Capturas 6 y 7: Tabla de Rutas y Asociación

Hemos configurado la tabla de rutas para la red pública añadiendo una regla que envía todo el tráfico externo (0.0.0.0/0) hacia el Internet Gateway. Después, **hemos asociado** manualmente esta tabla a la subred pública para confirmar que el enrutamiento funcione correctamente.





Captura 8: Asignación automática de IPs

Como el laboratorio reinicia las instancias cada 4 horas, **hemos activado** la opción de asignación automática de IP pública en la subred principal. De esta forma, **nos aseguramos** de que el servidor de acceso siempre reciba una IP pública nueva al arrancar sin tener que configurarla nosotros.



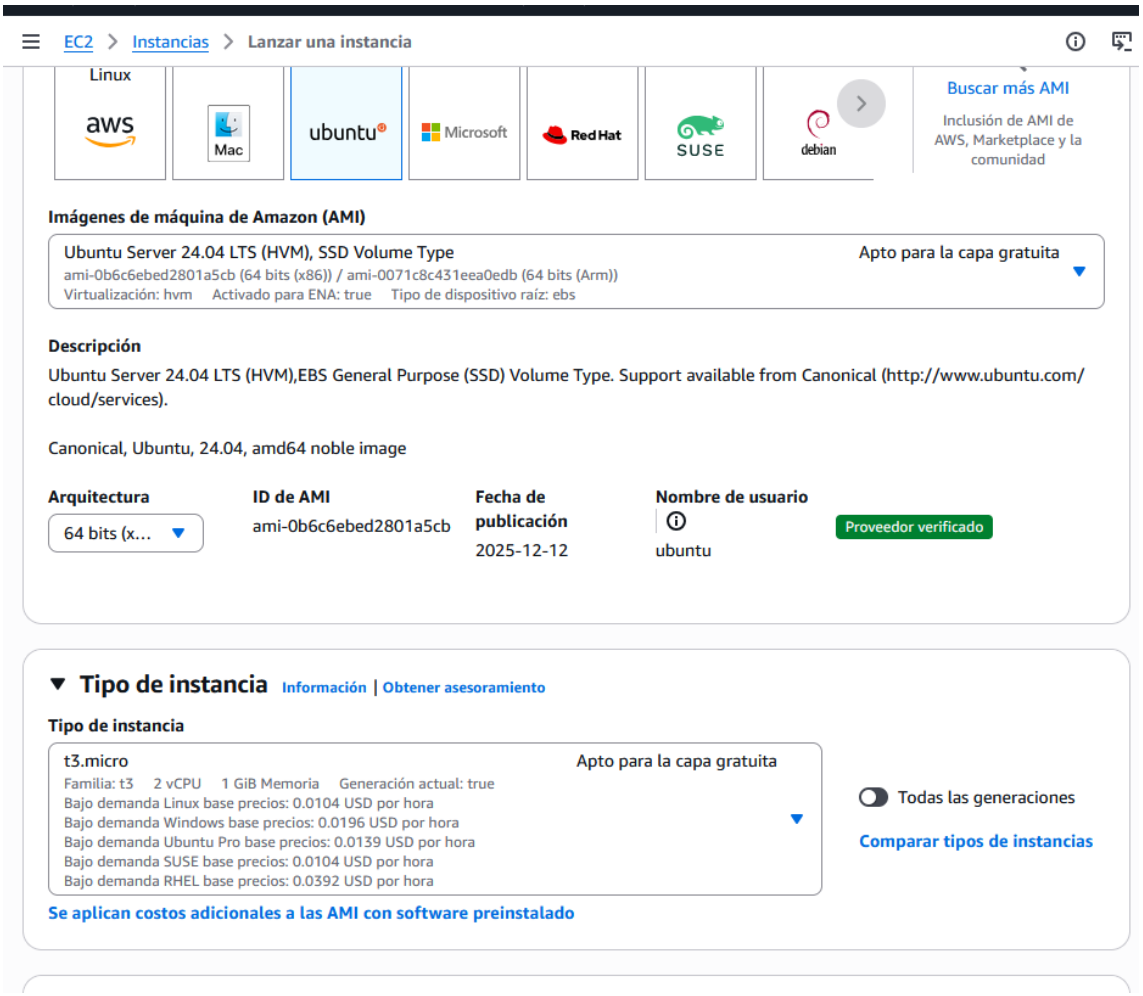
Bloque 2: Despliegue y Configuración de Instancias EC2

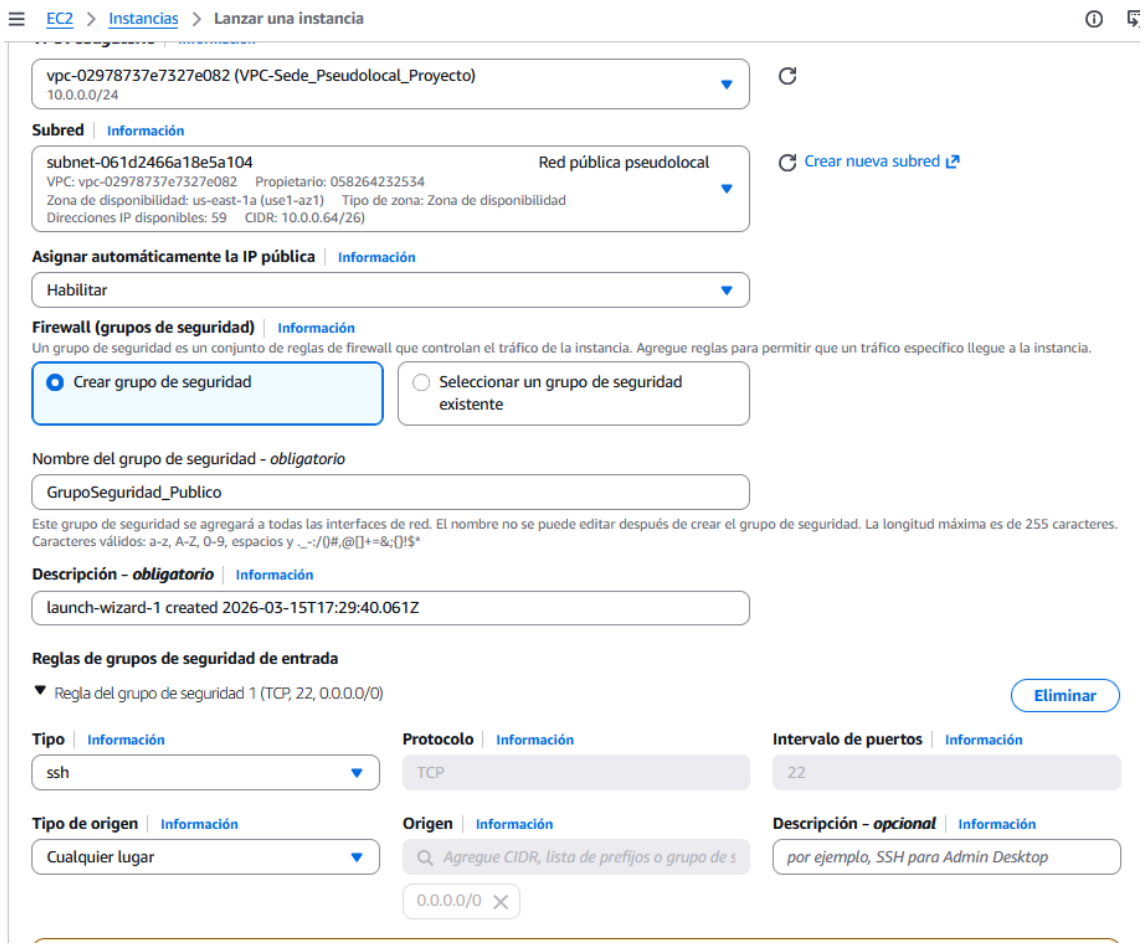
Una vez lista la red, hemos procedido a levantar los servidores con sus especificaciones correspondientes.

- **Selección de Sistemas y Hardware: Para el nodo de red, hemos elegido Ubuntu Server 24.04 LTS en una instancia t3.micro. Para el servidor de Windows, hemos optado por Windows Server 2016 Base en una instancia t3.small, asegurando que el Directorio Activo tenga suficiente memoria para operar.**
- **Seguridad y Aislamiento: Durante el lanzamiento, hemos asociado cada máquina a su subred; el Windows se ha configurado sin IP pública para que sea invisible desde**

internet. Hemos creado grupos de seguridad específicos abriendo solo los puertos necesarios (22 para SSH y 3389 para RDP interno).

- **Activación del Rol de Gateway: En el panel de AWS, hemos desactivado la "Comprobación de origen y destino" en el Ubuntu. Este ajuste es crítico, ya que hemos permitido que la instancia funcione como un router real, procesando tráfico de otras máquinas de la red.**




EC2 > Instancias > Lanzar una instancia
vpc-02978737e7327e082 (VPC-Sede_Pseudolocal_Proyecto)
10.0.0.0/24
Subred | Información
subnet-061d2466a18e5a104 Red pública pseudolocal
VPC: vpc-02978737e7327e082 Propietario: 058264232534
Zona de disponibilidad: us-east-1a (use1-az1) Tipo de zona: Zona de disponibilidad
Direcciones IP disponibles: 59 CIDR: 10.0.0.64/26)
Crear nueva subred
Asignar automáticamente la IP pública | Información
Habilitar
Firewall (grupos de seguridad) | Información
Un grupo de seguridad es un conjunto de reglas de firewall que controlan el tráfico de la instancia. Agregue reglas para permitir que un tráfico específico llegue a la instancia.
Crear grupo de seguridad
Seleccionar un grupo de seguridad existente
Nombre del grupo de seguridad - obligatorio
GrupoSeguridad_Publico
Este grupo de seguridad se agregará a todas las interfaces de red. El nombre no se puede editar después de crear el grupo de seguridad. La longitud máxima es de 255 caracteres.
Caracteres válidos: a-z, A-Z, 0-9, espacios y ._-:/()#,@[]+=&;{}!$*
Descripción - obligatorio | Información
launch-wizard-1 created 2026-03-15T17:29:40.061Z
Reglas de grupos de seguridad de entrada
Regla del grupo de seguridad 1 (TCP, 22, 0.0.0.0/0)
Eliminar
Tipo | Información
ssh
Protocolo | Información
TCP
Intervalo de puertos | Información
22
Tipo de origen | Información
Cualquier lugar
Origen | Información
Agregue CIDR, lista de prefijos o grupo de s
0.0.0.0/0
Descripción - opcional | Información
por ejemplo, SSH para Admin Desktop

▼ Imágenes de aplicaciones y sistemas operativos (Imagen de máquina de Amazon) Información

Una AMI posee el sistema operativo, el servidor de aplicaciones y las aplicaciones de la instancia. Si a continuación no ve una AMI adecuada, utilice el campo de búsqueda o elija **Buscar más AMI**.

Busque en nuestro catálogo completo que incluye miles de imágenes de sistemas operativos y aplicaciones

Recientes | Inicio rápido

Amazon Linux | macOS | Ubuntu | Windows | Red Hat | SUSE Linux | Debian

Buscar más AMI

Inclusión de AMI de AWS, Marketplace y la comunidad

Imágenes de máquina de Amazon (AMI)

Microsoft Windows Server 2016 Base — Apto para la capa gratuita

ami-0f258233417739216 (64 bits (x86))

Virtualización: hvm Activado para ENA: true Tipo de dispositivo raíz: ebs

Descripción

Microsoft Windows 2016 Datacenter edition. [English]

Microsoft Windows Server 2016 with Desktop Experience Locale English AMI provided by Amazon

Arquitectura	ID de AMI	Fecha de publicación	Nombre de usuario	
64 bits (x86)	ami-0f258233417739216	2026-02-12	Administrator	Proveedor verificado

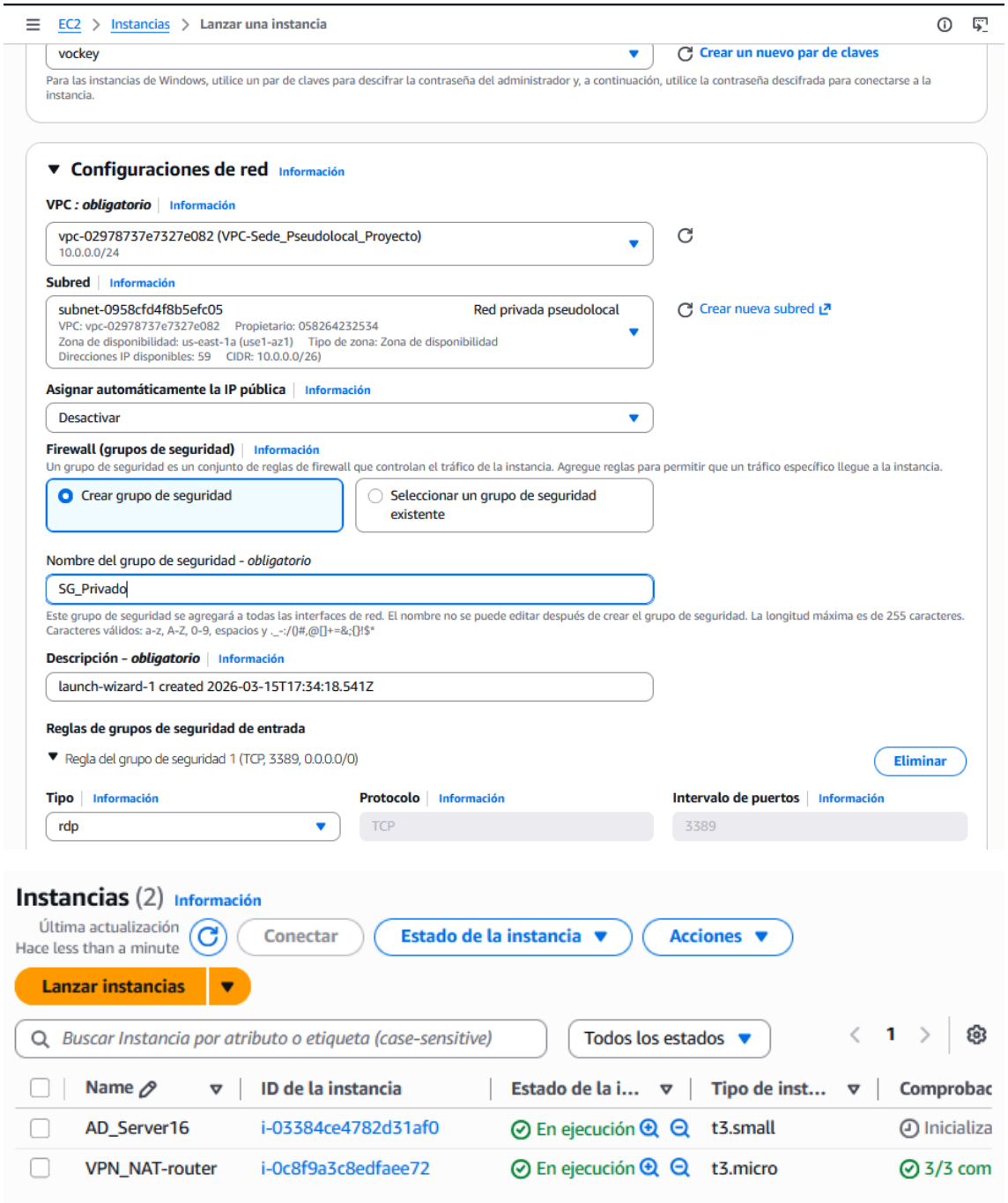
▼ Tipo de instancia Información | Obtener asesoramiento

Tipo de instancia

t3.small — Apto para la capa gratuita

Familia: t3 2 vCPU 2 GiB Memoria Generación actual: true

Todas las generaciones


EC2 > Instancias > Lanzar una instancia
vockey
Crear un nuevo par de claves
Para las instancias de Windows, utilice un par de claves para descifrar la contraseña del administrador y, a continuación, utilice la contraseña descifrada para conectarse a la instancia.
Configuraciones de red Información
VPC : obligatorio Información
vpc-02978737e7327e082 (VPC-Sede_Pseudolocal_Proyecto)
10.0.0.0/24
Subred Información
subnet-0958cfd4f8b5efc05 Red privada pseudolocal
VPC: vpc-02978737e7327e082 Propietario: 058264232534
Zona de disponibilidad: us-east-1a (use1-az1) Tipo de zona: Zona de disponibilidad
Direcciones IP disponibles: 59 CIDR: 10.0.0.0/26)
Crear nueva subred
Asignar automáticamente la IP pública Información
Desactivar
Firewall (grupos de seguridad) Información
Un grupo de seguridad es un conjunto de reglas de firewall que controlan el tráfico de la instancia. Agregue reglas para permitir que un tráfico específico llegue a la instancia.
Crear grupo de seguridad
Seleccionar un grupo de seguridad existente
Nombre del grupo de seguridad - obligatorio
SG_Privado
Este grupo de seguridad se agregará a todas las interfaces de red. El nombre no se puede editar después de crear el grupo de seguridad. La longitud máxima es de 255 caracteres.
Caracteres válidos: a-z, A-Z, 0-9, espacios y ._-:/()#,@[]+=&;{}!$*
Descripción - obligatorio Información
launch-wizard-1 created 2026-03-15T17:34:18.541Z
Reglas de grupos de seguridad de entrada
Regla del grupo de seguridad 1 (TCP, 3389, 0.0.0.0/0)
Eliminar
Tipo Información
rdp
Protocolo Información
TCP
Intervalo de puertos Información
3389

Instancias (2) Información
Última actualización
Hace less than a minute
Conectar
Estado de la instancia
Acciones
Lanzar instancias
Buscar Instancia por atributo o etiqueta (case-sensitive)
Todos los estados
1
Name
ID de la instancia
Estado de la i...
Tipo de inst...
Comprobac
AD_Server16
i-03384ce4782d31af0
En ejecución
t3.small
Inicializa
VPN_NAT-router
i-0c8f9a3c8edfaee72
En ejecución
t3.micro
3/3 com



Acceso mediante Túnel SSH (Local Port Forwarding)

Para administrar el **Windows Server**, que se encuentra aislado en una subred privada, utilizamos el servidor **Ubuntu** como puente. El comando utilizado es:

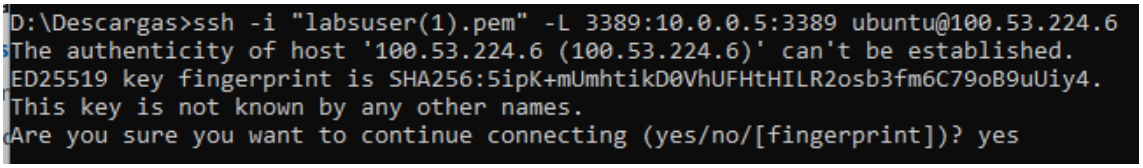
ssh -i "labsuser(1).pem" -L 3390:10.0.0.50:3389 ubuntu@100.53.224.6

¿Qué hace este comando?

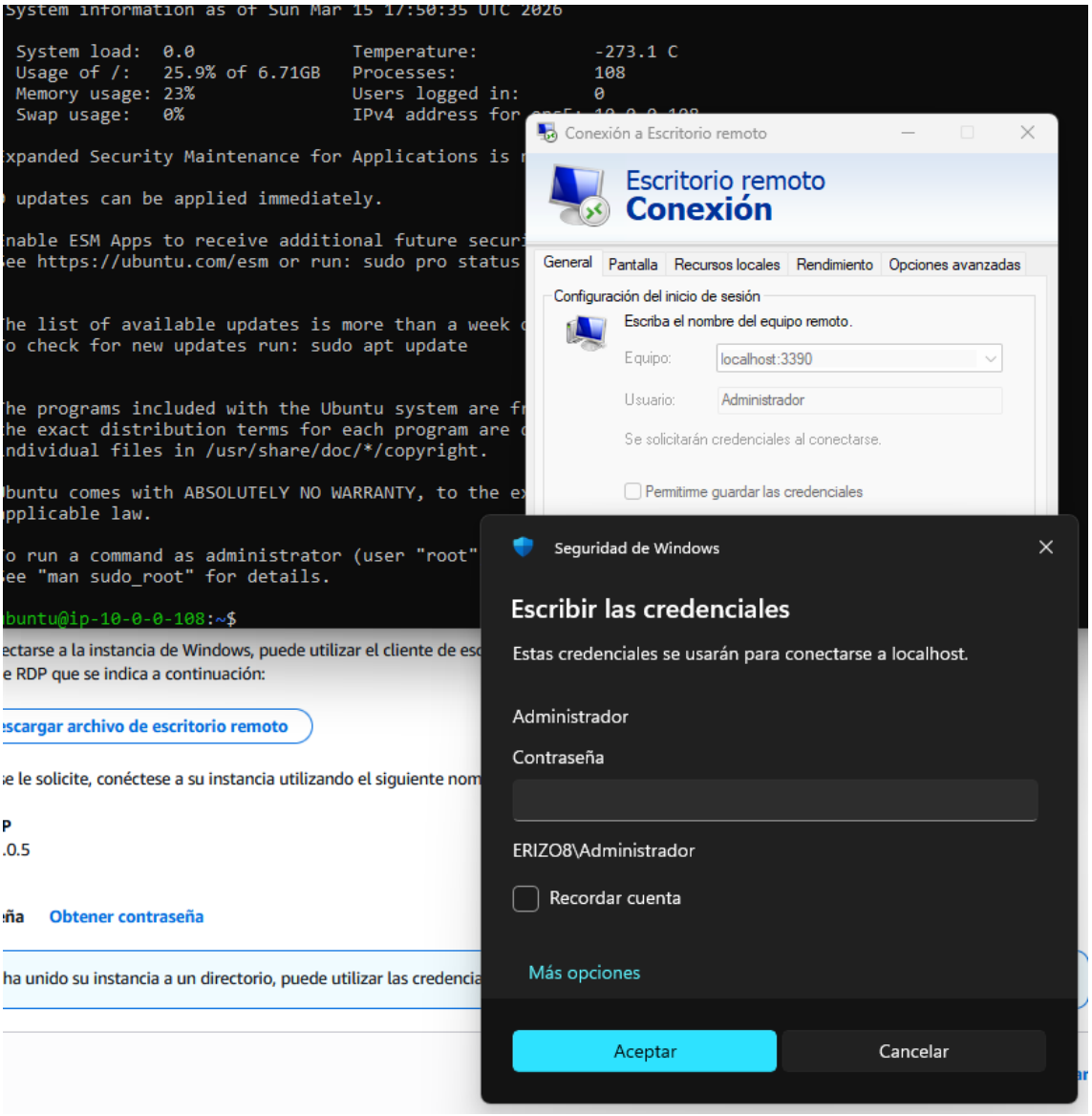
- **Encapsulado seguro:** Usa nuestra clave privada para abrir una sesión cifrada en el Ubuntu
- **Mapeo de puertos (-L):** Redirige el tráfico de nuestro puerto local **3390** hacia el puerto **3389** (RDP) del Windows interno (**10.0.0.50**).
- **Salto de red:** El tráfico viaja a través de la IP pública del router/VPN (100.53.224.6) para saltar la barrera de la subred privada

Justificación del diseño

1. **Seguridad (Bastion Host):** El servidor Windows no tiene IP pública, por lo que es inmune a ataques directos desde internet.
2. **Mantenimiento:** Dado que el laboratorio reinicia las instancias cada 4 horas, este túnel nos permite recuperar el control del entorno de forma inmediata simplemente actualizando la IP pública si esta cambia.



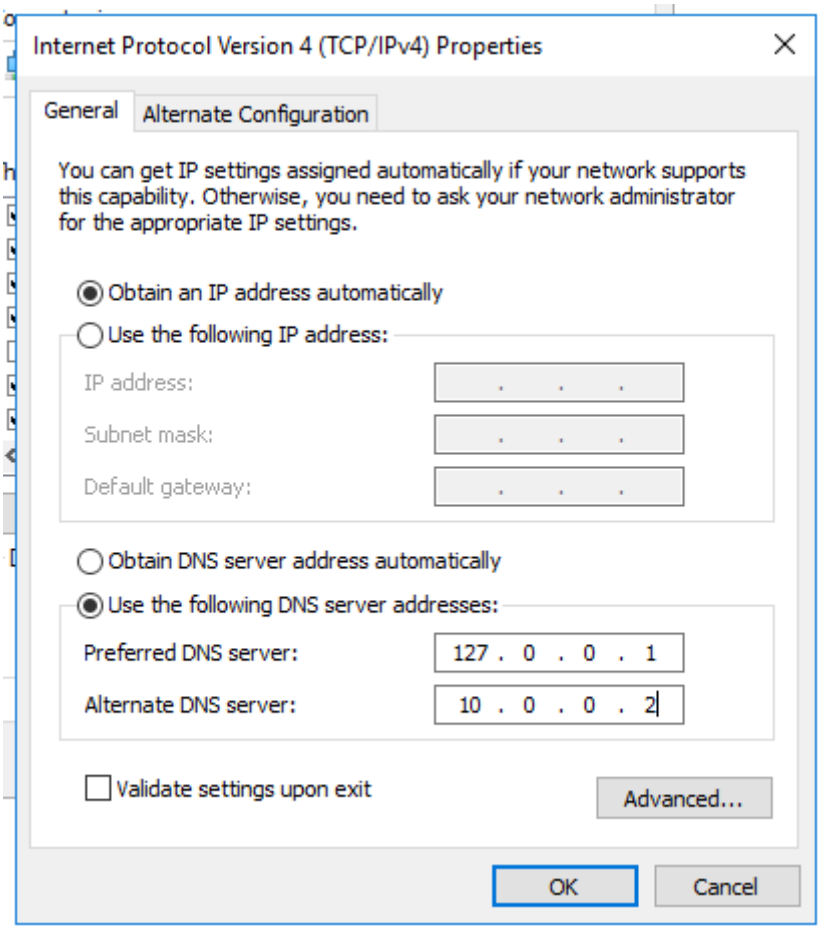
Conexión RDP: Utilizando la aplicación de Escritorio Remoto, **nos hemos conectado** a localhost:3390. Esto **nos ha permitido** gestionar el servidor privado de forma transparente a través del túnel cifrado que **hemos abierto** previamente.



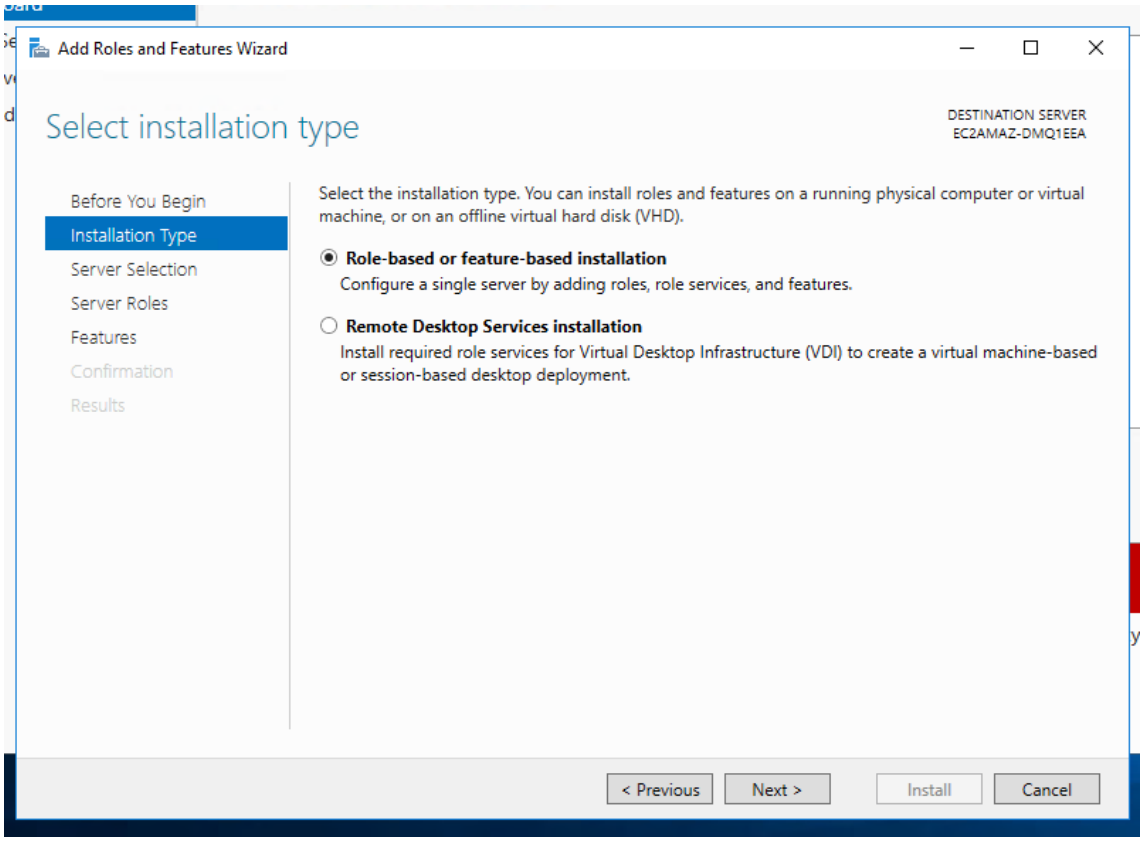
Bloque 4: Implementación del Directorio Activo y DNS

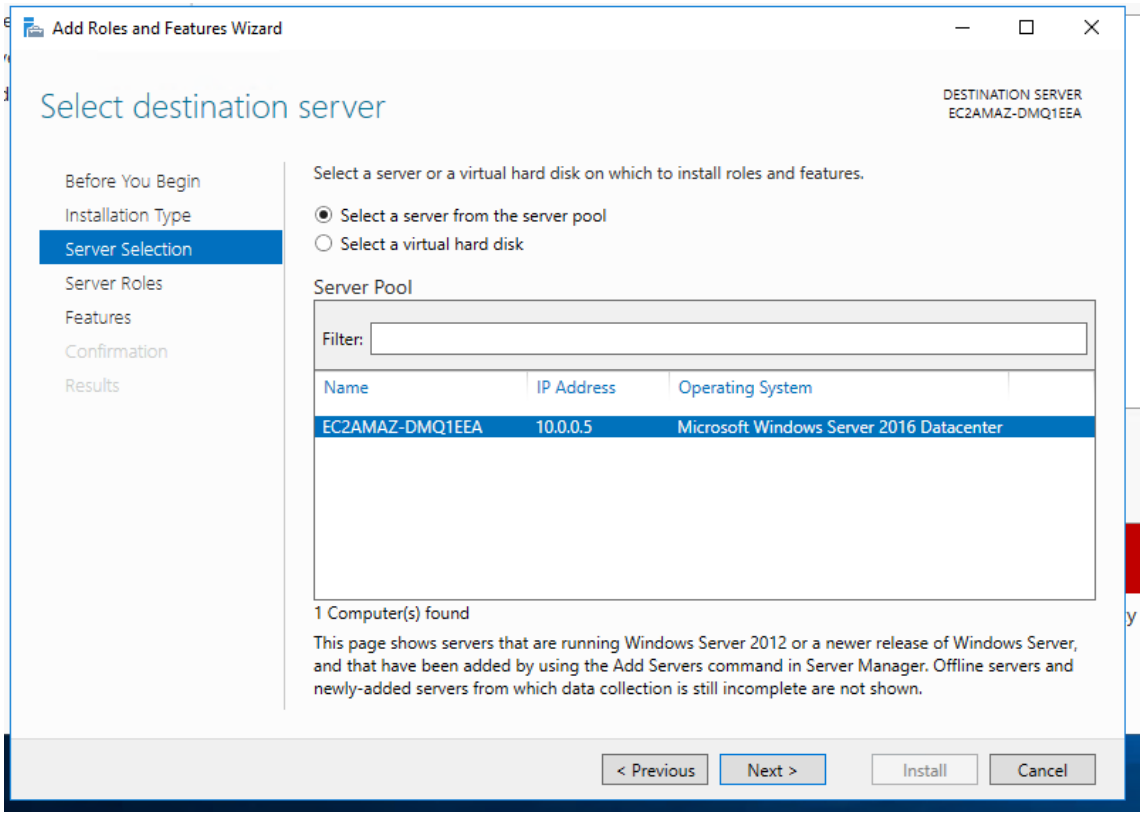
Con el acceso garantizado, **hemos convertido** el Windows Server en el núcleo de nuestra red.

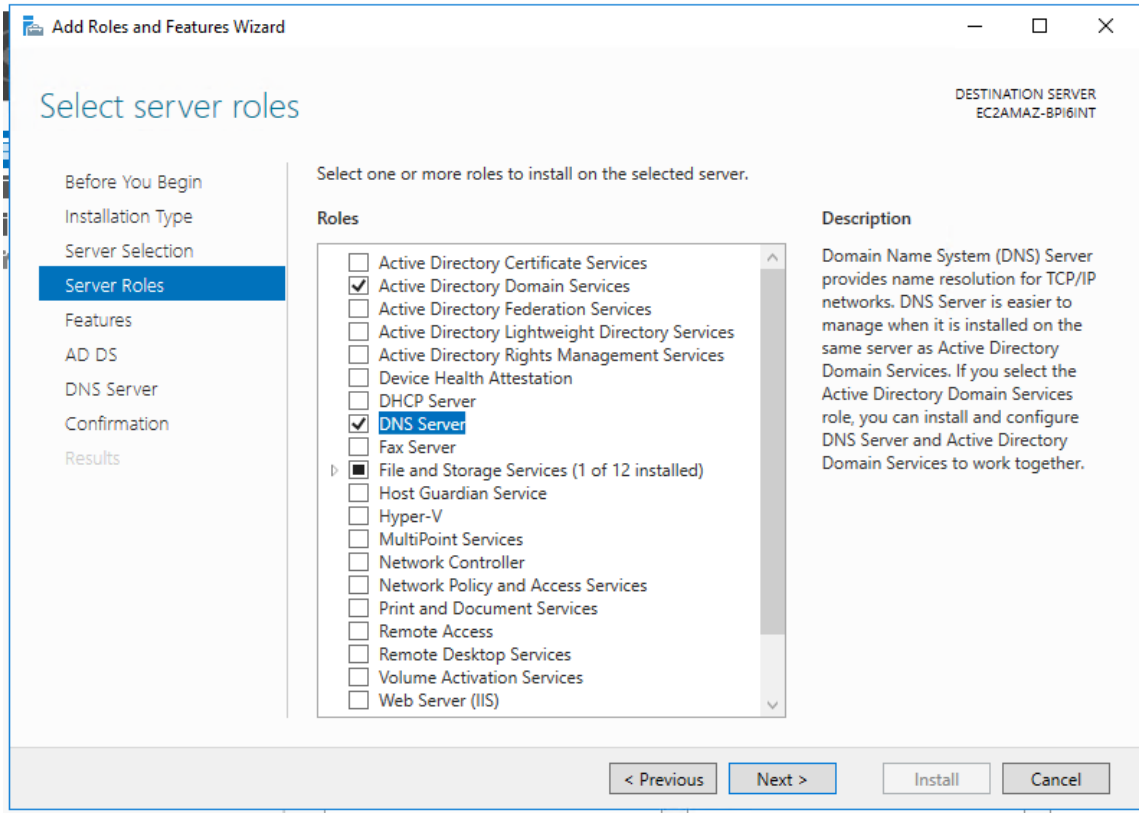
- **Configuración de Red Interna**: **Hemos ajustado** las propiedades de IPv4 para que el servidor use 127.0.0.1 como DNS primario, requisito indispensable para el correcto funcionamiento de los servicios de dominio.

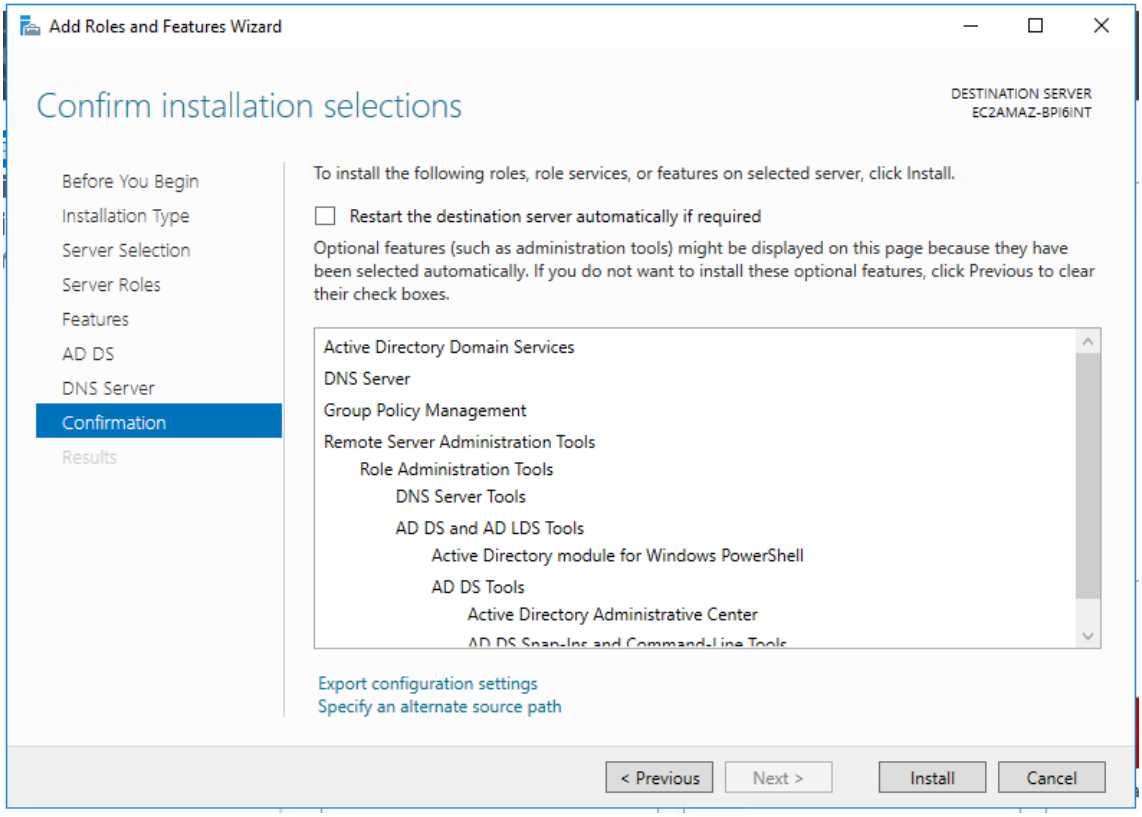


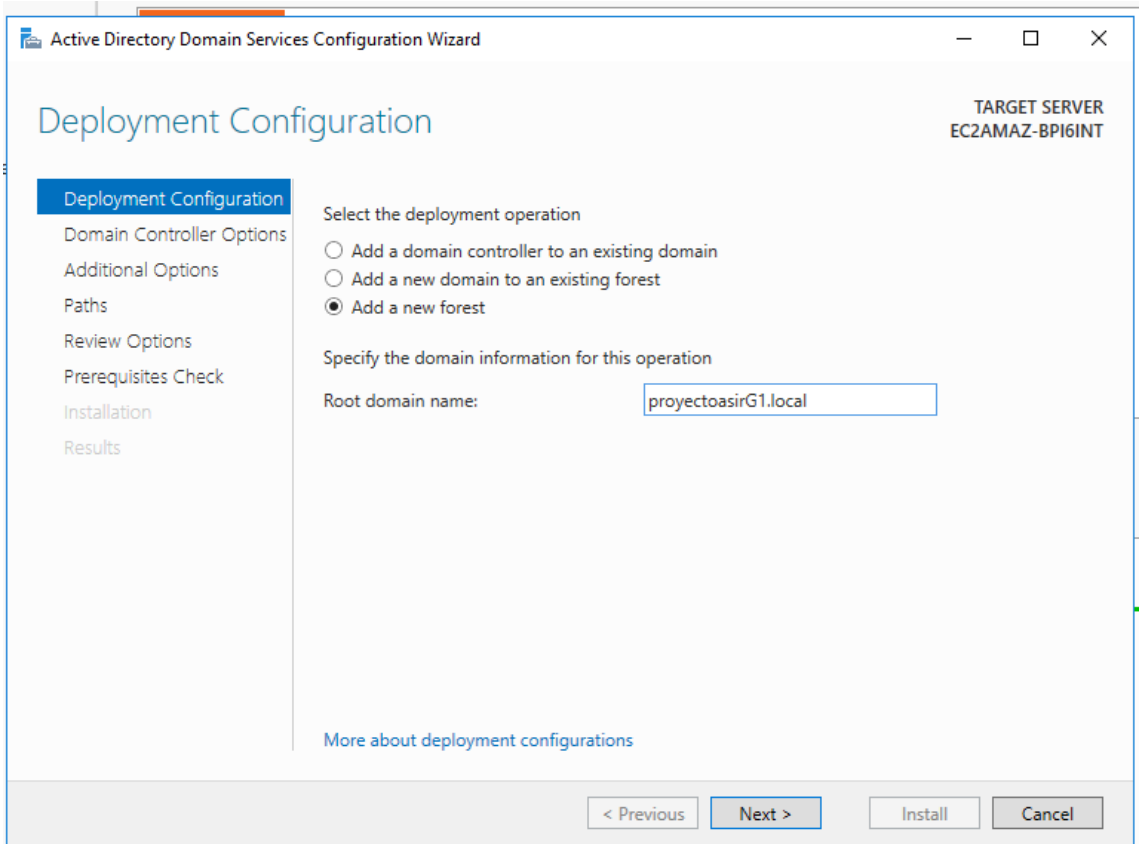
Promoción del Dominio: **Hemos instalado** los roles de **Active Directory Domain Services** y **DNS Server**. **Hemos configurado** un nuevo bosque llamado **proyectoasirG1.local**, verificando que el sistema superara todos los requisitos previos de seguridad y compatibilidad.

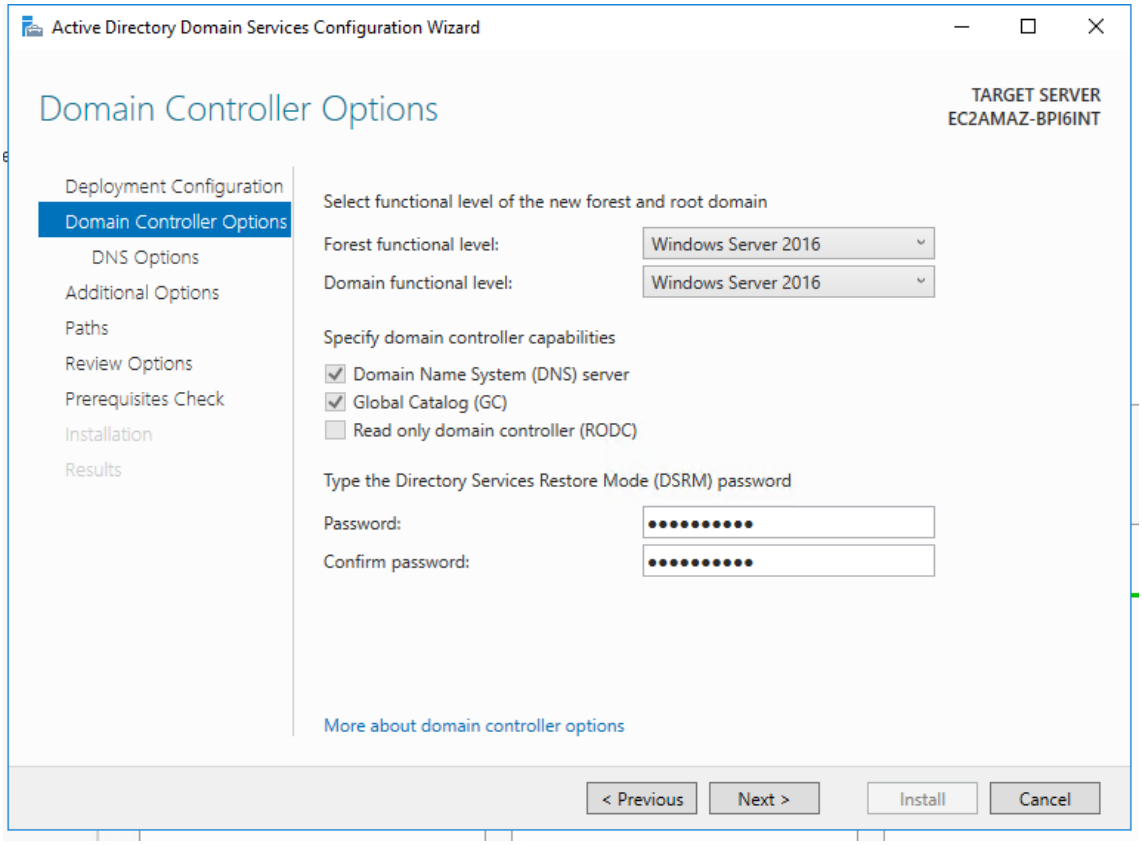

Add Roles and Features Wizard
Select installation type
DESTINATION SERVER
EC2AMAZ-DMQ1EEA
Before You Begin
Installation Type
Server Selection
Server Roles
Features
Confirmation
Results
Select the installation type. You can install roles and features on a running physical computer or virtual machine, or on an offline virtual hard disk (VHD).
Role-based or feature-based installation
Configure a single server by adding roles, role services, and features.
Remote Desktop Services installation
Install required role services for Virtual Desktop Infrastructure (VDI) to create a virtual machine-based or session-based desktop deployment.
< Previous
Next >
Install
Cancel

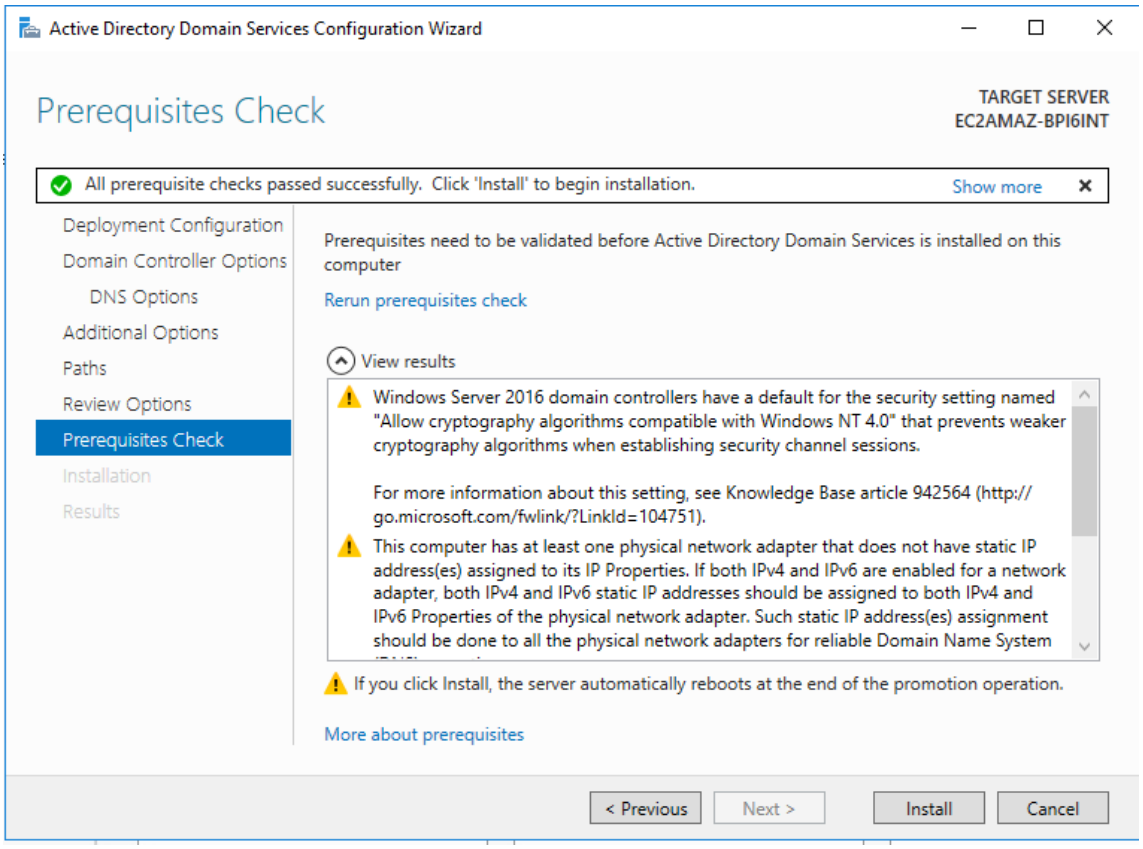

Add Roles and Features Wizard
Select destination server
DESTINATION SERVER
EC2AMAZ-DMQ1EEA
Before You Begin
Installation Type
Server Selection
Server Roles
Features
Confirmation
Results
Select a server or a virtual hard disk on which to install roles and features.
Select a server from the server pool
Select a virtual hard disk
Server Pool
Filter:
Name
IP Address
Operating System
EC2AMAZ-DMQ1EEA
10.0.0.5
Microsoft Windows Server 2016 Datacenter
1 Computer(s) found
This page shows servers that are running Windows Server 2012 or a newer release of Windows Server, and that have been added by using the Add Servers command in Server Manager. Offline servers and newly-added servers from which data collection is still incomplete are not shown.
< Previous
Next >
Install
Cancel


Add Roles and Features Wizard
Select server roles
DESTINATION SERVER
EC2AMAZ-BPI6INT
Before You Begin
Installation Type
Server Selection
Server Roles
Features
AD DS
DNS Server
Confirmation
Results
Select one or more roles to install on the selected server.
Roles
Active Directory Certificate Services
Active Directory Domain Services
Active Directory Federation Services
Active Directory Lightweight Directory Services
Active Directory Rights Management Services
Device Health Attestation
DHCP Server
DNS Server
Fax Server
File and Storage Services (1 of 12 installed)
Host Guardian Service
Hyper-V
MultiPoint Services
Network Controller
Network Policy and Access Services
Print and Document Services
Remote Access
Remote Desktop Services
Volume Activation Services
Web Server (IIS)
Description
Domain Name System (DNS) Server provides name resolution for TCP/IP networks. DNS Server is easier to manage when it is installed on the same server as Active Directory Domain Services. If you select the Active Directory Domain Services role, you can install and configure DNS Server and Active Directory Domain Services to work together.
< Previous
Next >
Install
Cancel

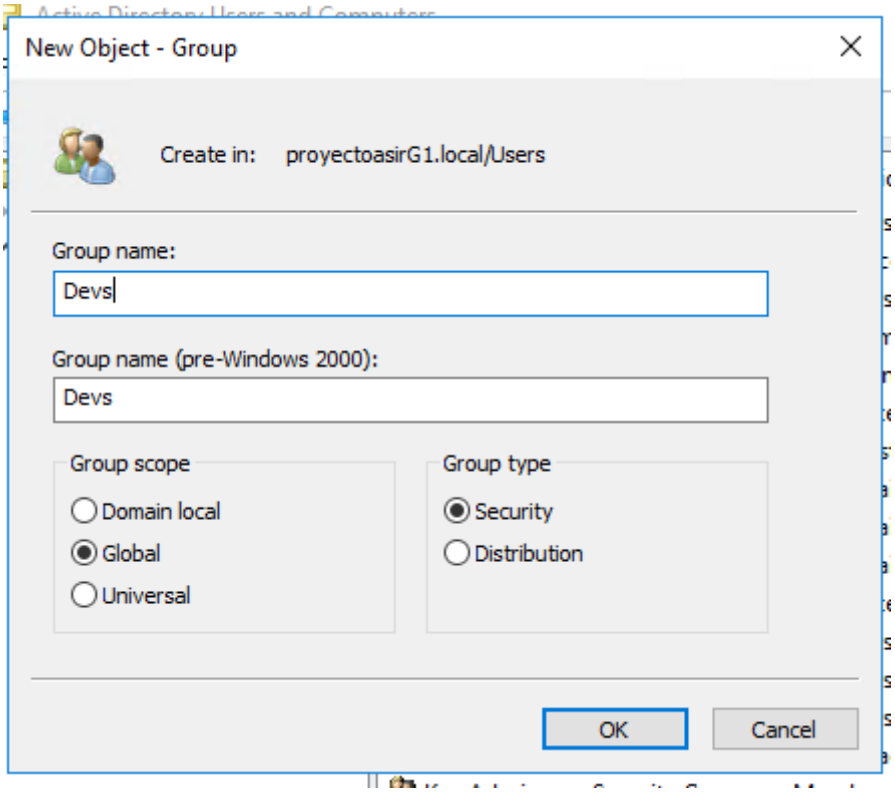

Add Roles and Features Wizard
Confirm installation selections
DESTINATION SERVER
EC2AMAZ-BPI6INT
Before You Begin
Installation Type
Server Selection
Server Roles
Features
AD DS
DNS Server
Confirmation
Results
To install the following roles, role services, or features on selected server, click Install.
Restart the destination server automatically if required
Optional features (such as administration tools) might be displayed on this page because they have been selected automatically. If you do not want to install these optional features, click Previous to clear their check boxes.
Active Directory Domain Services
DNS Server
Group Policy Management
Remote Server Administration Tools
Role Administration Tools
DNS Server Tools
AD DS and AD LDS Tools
Active Directory module for Windows PowerShell
AD DS Tools
Active Directory Administrative Center
Export configuration settings
Specify an alternate source path
< Previous
Next >
Install
Cancel

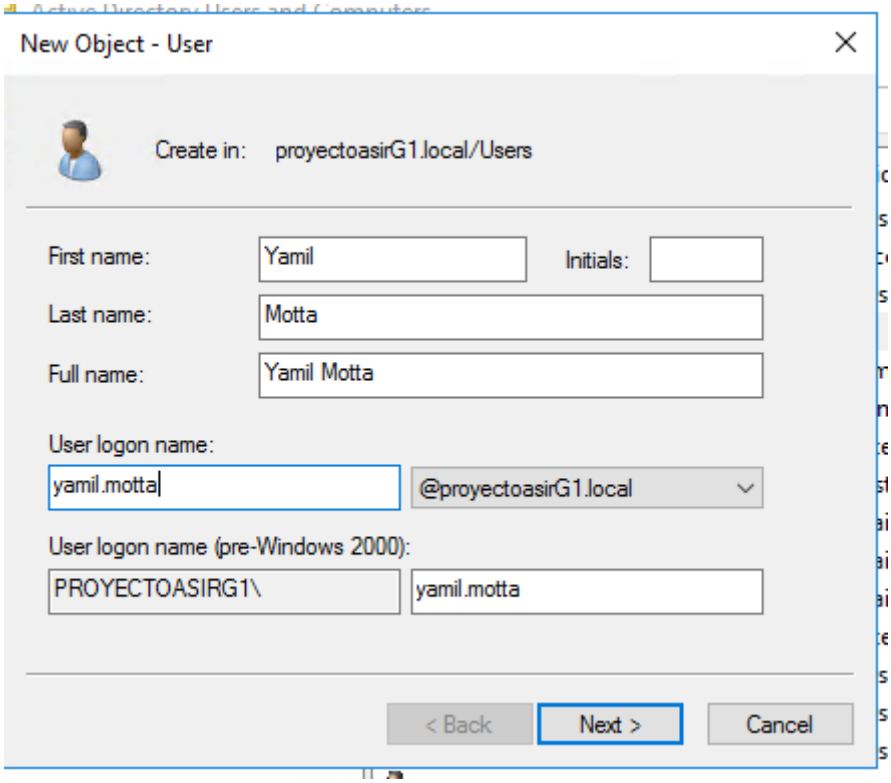

Active Directory Domain Services Configuration Wizard
Deployment Configuration
TARGET SERVER
EC2AMAZ-BPI6INT
Deployment Configuration
Domain Controller Options
Additional Options
Paths
Review Options
Prerequisites Check
Installation
Results
Select the deployment operation
Add a domain controller to an existing domain
Add a new domain to an existing forest
Add a new forest
Specify the domain information for this operation
Root domain name:
proyectoasirG1.local
More about deployment configurations
< Previous
Next >
Install
Cancel

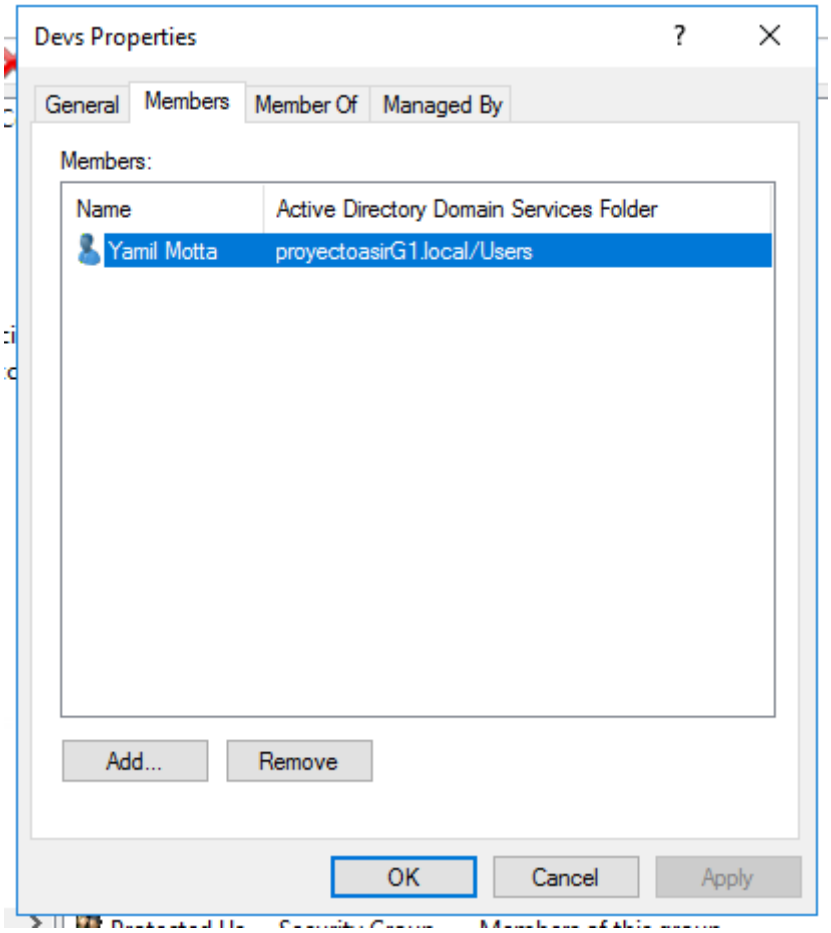

Active Directory Domain Services Configuration Wizard
Domain Controller Options
TARGET SERVER
EC2AMAZ-BPI6INT
Deployment Configuration
Domain Controller Options
DNS Options
Additional Options
Paths
Review Options
Prerequisites Check
Installation
Results
Select functional level of the new forest and root domain
Forest functional level:
Windows Server 2016
Domain functional level:
Windows Server 2016
Specify domain controller capabilities
Domain Name System (DNS) server
Global Catalog (GC)
Read only domain controller (RODC)
Type the Directory Services Restore Mode (DSRM) password
Password:
Confirm password:
More about domain controller options
< Previous
Next >
Install
Cancel



Estructura Organizativa: Para validar la instalación, **hemos creado** grupos de seguridad (Devs) y usuarios de prueba (Yamil Motta) dentro de la consola de Usuarios y Equipos de Active Directory, dejando el entorno listo para la gestión de políticas.




New Object - User
Create in: proyectoasirG1.local/Users
First name: Yamil
Initials:
Last name: Motta
Full name: Yamil Motta
User logon name:
yamil.motta
@proyectoasirG1.local
User logon name (pre-Windows 2000):
PROYECTOASIRG1\
yamil.motta
< Back
Next >
Cancel

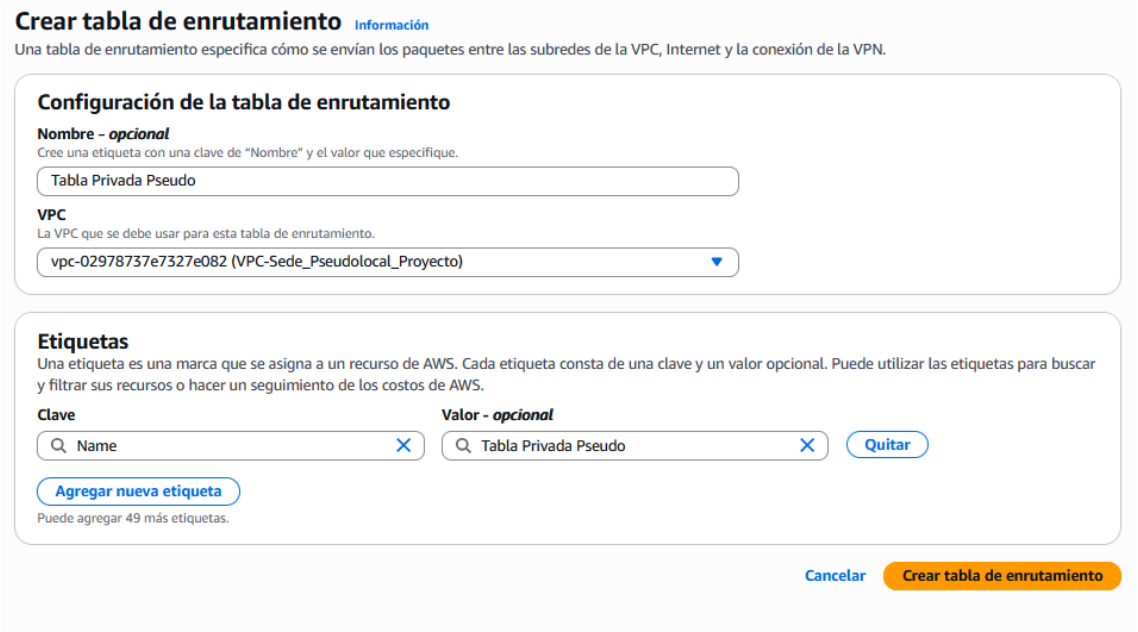

Devs Properties
General
Members
Member Of
Managed By
Members:
Name
Active Directory Domain Services Folder
Yamil Motta
proyectoasirG1.local/Users
Add...
Remove
OK
Cancel
Apply

Bloque 5: Implementación de VPN (WireGuard) y Persistencia (DDNS)

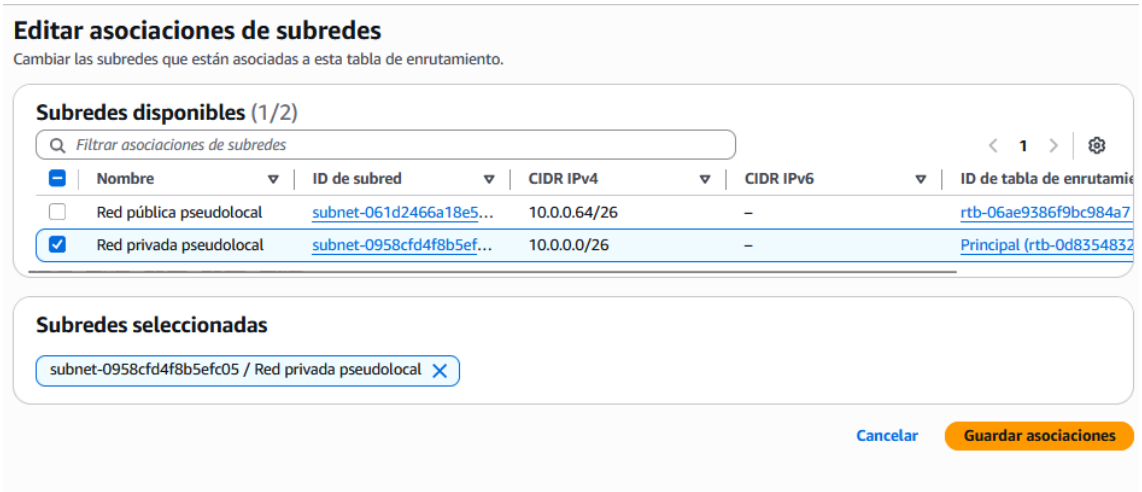
Para superar la limitación del acceso mediante túneles SSH temporales y lograr una conectividad profesional con la subred privada, hemos desplegado un servidor VPN WireGuard en el nodo Ubuntu.

- **Configuración y Seguridad:** Se ha establecido un túnel cifrado (10.100.0.0/24) y, como medida de seguridad adicional, se ha modificado el puerto por defecto, abriendo el puerto UDP 55000 en el Security Group de AWS exclusivamente para este tráfico.
- **Persistencia y Automatización (DDNS):** Dado que el entorno de laboratorio reinicia las instancias cada 4 horas alterando la IP pública de la pasarela, hemos integrado el servicio DuckDNS. Mediante un script bash y la programación de una tarea en el sistema (crontab), el servidor actualiza automáticamente su dominio dinámico cada 5 minutos. Esto garantiza que la conexión VPN sea persistente y sobreviva a los reinicios sin intervención manual.
- **Validación Final:** Al activar el túnel, el equipo cliente local pasa a formar parte de la red privada de AWS. Como se demuestra en la última captura, esto nos permite gestionar el Directorio Activo mediante una conexión RDP directa a la IP 10.0.0.50, cumpliendo con éxito todos los objetivos de arquitectura y seguridad de esta primera fase

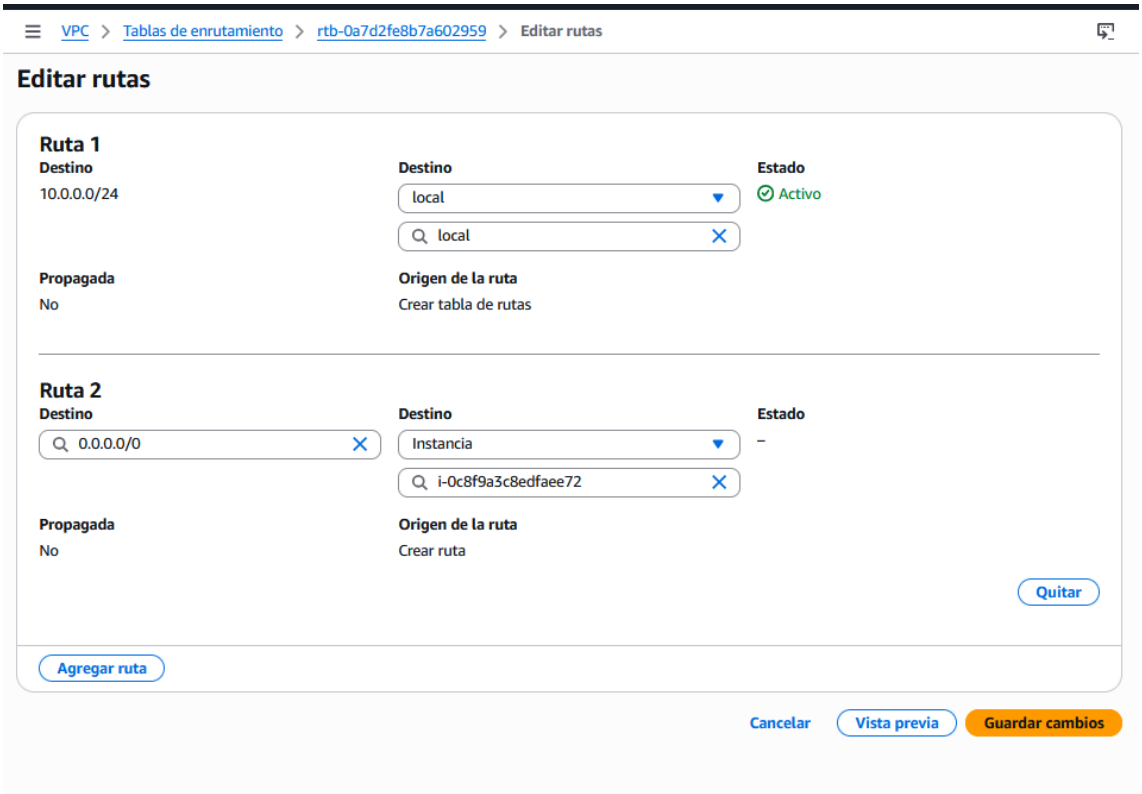
Creación de la tabla de enrutamiento personalizada en AWS (Tabla Privada Pseudo), paso fundamental para controlar el tráfico de la zona segura



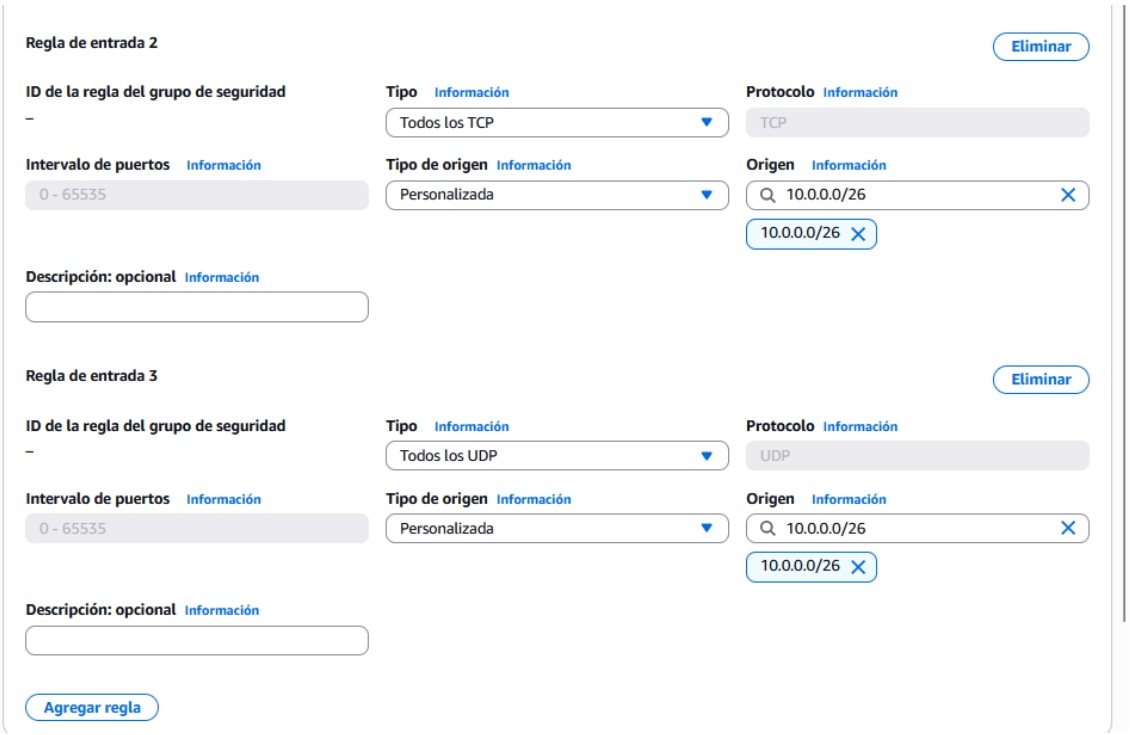
Asociación explícita de la nueva tabla de rutas con la subred Red privada pseudolocal, asegurando que el Windows Server siga estas reglas



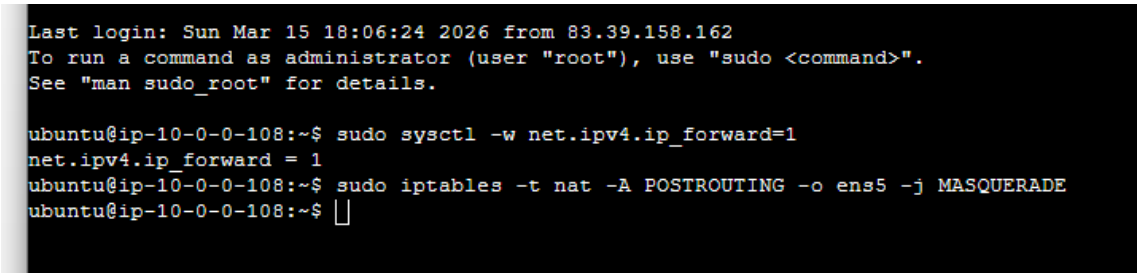
Configuración de la ruta por defecto (0.0.0.0/0) apuntando a la instancia EC2 del Ubuntu (VPN_NAT-router), estableciendo así el camino de salida.



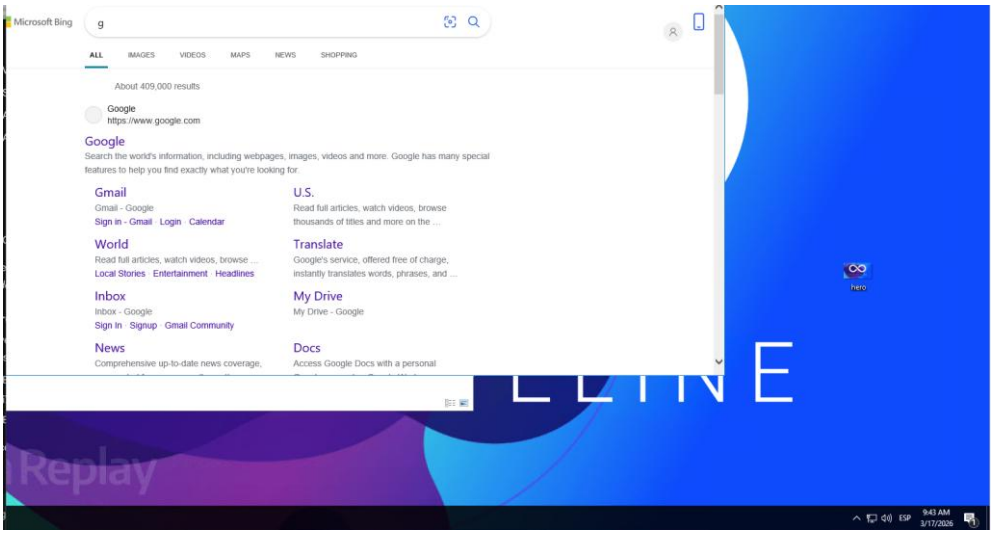
Modificación del Grupo de Seguridad del servidor Ubuntu para permitir todo el tráfico entrante (TCP/UDP) originado desde la subred privada 10.0.0.0/26



Activación del reenvío de paquetes (ip_forward) y configuración de las reglas de enmascaramiento NAT (MASQUERADE) mediante iptables en Linux



Prueba de conectividad exitosa cargando páginas web desde el servidor Windows aislado, confirmando el correcto funcionamiento del enrutamiento y el NAT



Archivo de configuración inicial del servidor WireGuard (wg0.conf) en Ubuntu, definiendo el rango de IPs del túnel (10.100.0.1/24) y las reglas de red

```
 GNU nano 7.2                                 /etc/wireguard/wg0.conf *
[Interface]
# La IP del Ubuntu dentro de la VPN (el /24 nos da hasta 254 IPs libres para clientes)
Address = 10.100.0.1/24
ListenPort = 55000
PrivateKey = 4HIAdpNjdWO5Nl3z508a1fST2E6L+WV2m6cIzQkx8Xo=

# Reglas
PostUp = iptables -A FORWARD -i wg0 -j ACCEPT; iptables -t nat -A POSTROUTING -o ens5 -j MASQUERADE
PostDown = iptables -D FORWARD -i wg0 -j ACCEPT; iptables -t nat -D POSTROUTING -o ens5 -j MASQUERADE

# --- A PARTIR DE AQUÍ AÑADIREMOS LAS 4 CONEXIONES (PEERS) ---
```

Apertura del puerto personalizado 55000 (UDP) en el Grupo de Seguridad de AWS del Ubuntu para permitir las conexiones externas de la VPN



Inclusión de la clave pública del equipo cliente ("Peer") en la configuración del servidor VPN para autorizar su acceso al túnel(equipo de prueba)

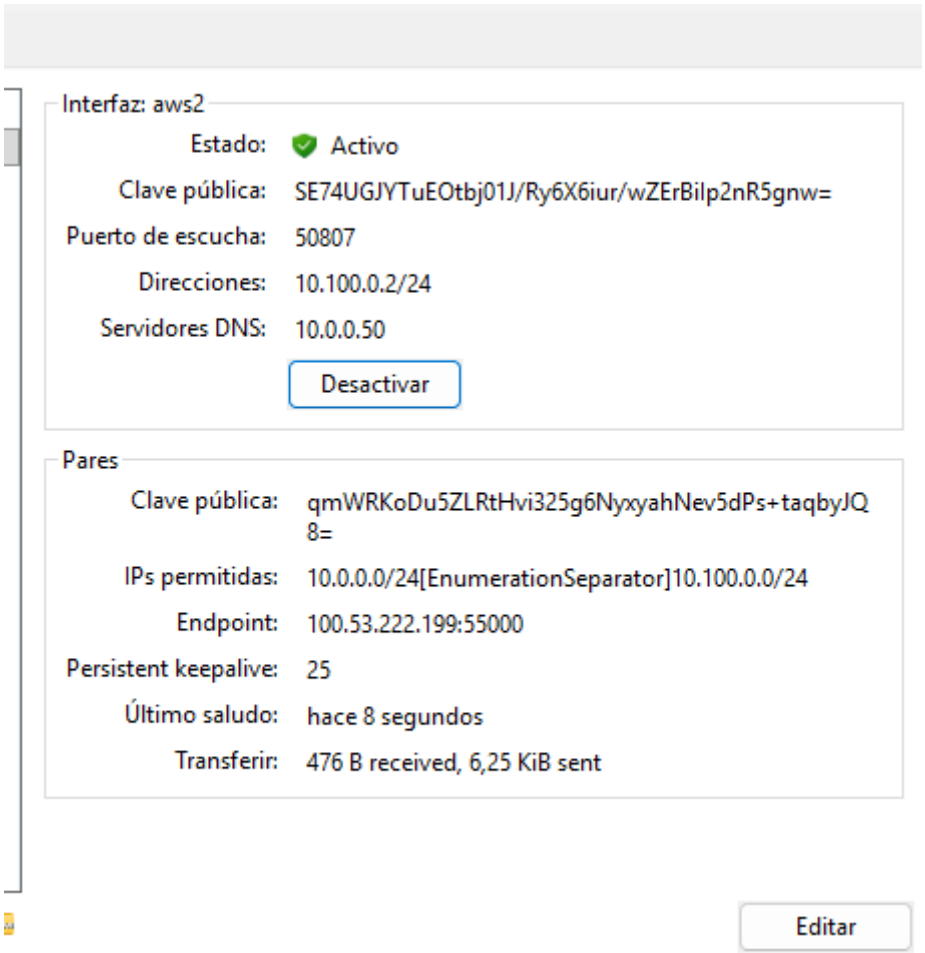
Nota: En un entorno de producción real, la clave privada jamás debe ser expuesta

```
 GNU nano 7.2                                 /etc/wireguard/wg0.conf *
[Interface]
# La IP del Ubuntu dentro de la VPN (el /24 nos da hasta 254 IPs libres para clientes)
Address = 10.100.0.1/24
ListenPort = 55000
PrivateKey = 4HIAdpNjdWO5Nl3z508a1fST2E6L+WV2m6cIzQkx8Xo=

# Reglas
PostUp = iptables -A FORWARD -i wg0 -j ACCEPT; iptables -t nat -A POSTROUTING -o ens5 -j MASQUERADE
PostDown = iptables -D FORWARD -i wg0 -j ACCEPT; iptables -t nat -D POSTROUTING -o ens5 -j MASQUERADE

# --- A PARTIR DE AQUÍ AÑADIREMOS LAS 4 CONEXIONES (PEERS) ---
[Peer]
# PC de Casa (Carlos Test)
PublicKey = ftsmFseYDs3DVNaQWU/WgRsDjMv8iQyKTV6t5zAslAk=
AllowedIPs = 10.100.0.2/32
```

Cliente local de WireGuard en Windows configurado y emparejado exitosamente utilizando la IP pública temporal del servidor



Comprobación inicial de conectividad ICMP (Ping) desde el equipo físico local hacia la IP privada del router Ubuntu (10.0.0.108) a través del túnel VPN

```
C:\Users\Carlos>ping 10.0.0.108

Haciendo ping a 10.0.0.108 con 32 bytes de datos:
Respuesta desde 10.0.0.108: bytes=32 tiempo=100ms TTL=64
Respuesta desde 10.0.0.108: bytes=32 tiempo=100ms TTL=64
Respuesta desde 10.0.0.108: bytes=32 tiempo=100ms TTL=64
Respuesta desde 10.0.0.108: bytes=32 tiempo=100ms TTL=64

Estadísticas de ping para 10.0.0.108:
    Paquetes: enviados = 4, recibidos = 4, perdidos = 0
    (0% perdidos),
Tiempos aproximados de ida y vuelta en milisegundos:
    Mínimo = 100ms, Máximo = 100ms, Media = 100ms

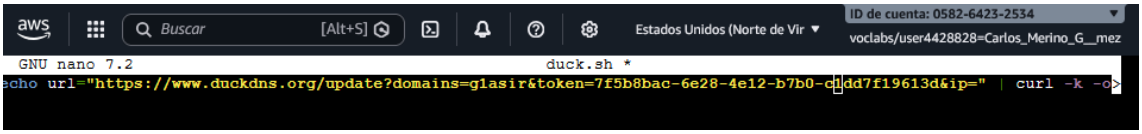
C:\Users\Carlos>
```

Persistencia de la conexión (DuckDNS)

Registro del dominio dinámico en DuckDNS (g1asir.duckdns.org), solución implementada para mitigar la rotación de la IP pública del laboratorio



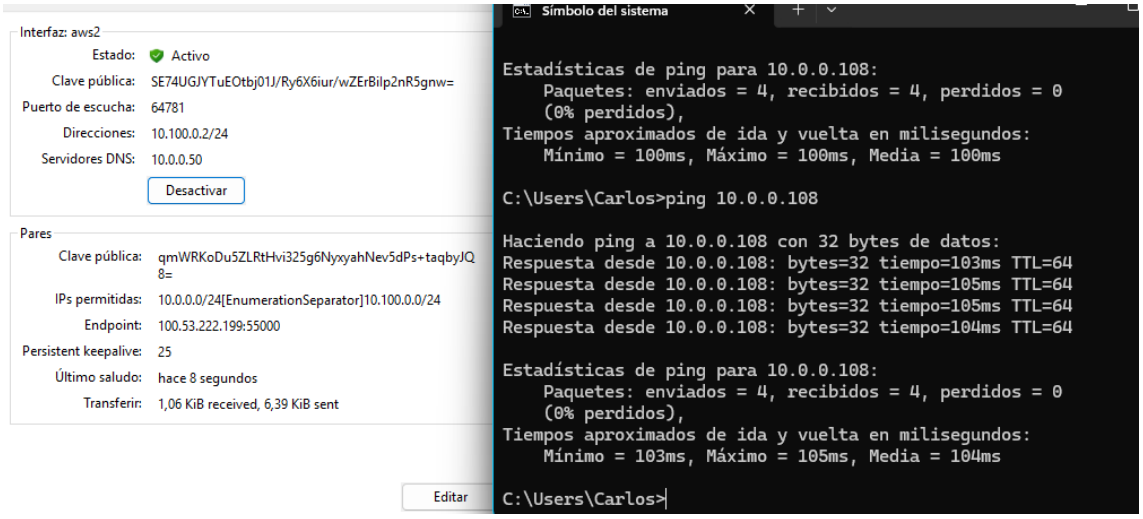
Creación del script ejecutable en bash (duck.sh) encargado de reportar la IP pública actual de la instancia de AWS al servicio de DuckDNS



Programación de tareas del sistema en Linux (crontab) para automatizar la actualización de la IP en DuckDNS cada 5 minutos de forma silenciosa

```
  GNU nano 7.2                                  /tmp/crontab.saUVEW/crontab
# Edit this file to introduce tasks to be run by cron.
#
# Each task to run has to be defined through a single line
# indicating with different fields when the task will be run
# and what command to run for the task
#
# To define the time you can provide concrete values for
# minute (m), hour (h), day of month (dom), month (mon),
# and day of week (dow) or use '*' in these fields (for 'any').
#
# Notice that tasks will be started based on the cron's system
# daemon's notion of time and timezones.
#
# Output of the crontab jobs (including errors) is sent through
# email to the user the crontab file belongs to (unless redirected).
#
# For example, you can run a backup of all your user accounts
# at 5 a.m every week with:
# 0 5 * * 1 tar -zcf /var/backups/home.tgz /home/
#
# For more information see the manual pages of crontab(5) and cron(8)
#
# m h  dom mon dow   command
*/5 * * * * ~/duckdns/duck.sh >/dev/null 2>&1
```

Verificación de la estabilidad del túnel VPN y confirmación de recepción de paquetes tras aplicar las automatizaciones en el servidor.



Actualización del cliente WireGuard local para utilizar el nuevo dominio dinámico configurado en lugar de la IP efímera que caduca tras los reinicios

Interfaz: aws2

Estado:	Inactivo
Clave pública:	SE74UGJYTuEOtbj01J/Ry6X6iur/wZErBilp2nR5gnw=
Direcciones:	10.100.0.2/24
Servidores DNS:	10.0.0.50

Activar

Pares

Clave pública:	qmWRKoDu5ZLRtHvi325g6NyxyahNev5dPs+taqbyJQ8=
IPs permitidas:	10.0.0.0/24[EnumerationSeparator]10.100.0.0/24
Endpoint:	g1asir.duckdns.org:55000
Persistent keepalive:	25

Al estar conectado a la VPN se puede acceder al Windows Server 2016 sin problemas:

